

hw4-l-u-s-t-buster (/github/BEE4750-FA24/hw4-l-u-s-t-buster/tree/Fall24)

/ hw04.ipynb (/github/BEE4750-FA24/hw4-l-u-s-t-buster/tree/Fall24/hw04.ipynb)

BEE 4750 Homework 4: Linear Programming and Capacity Expansion

Name: Fenya Bartram

ID: fdb34

Due Date

Thursday, 11/07/23, 9:00pm

Overview

Instructions

- Problem 1 asks you to formulate and solve a resource allocation problem using linear programming.
- Problem 2 asks you to formulate, solve, and analyze a standard generating capacity expansion problem.
- Problem 3 (5750 only) asks you to add a CO₂ constraint to the capacity expansion problem and identify changes in the resulting solution.

Load Environment

The following code loads the environment and makes sure all needed packages are installed. This should be at the start of most Julia scripts.

```
In [1]: import Pkg
        Pkg.activate(@__DIR__)
        Pkg.instantiate()
```

Activating project at `c:\Users\fenya\Documents\BEE4750\hw4-l-u-s-t-buster`

```
In [2]: # import Pkg; Pkg.add("NamedArrays")
```

```
In [3]: using JuMP
using HiGHS
using DataFrames
using Plots
using Measures
using CSV
using MarkdownTables
using NamedArrays
```

Problems (Total: 50/60 Points)

Problem 1 (20 points)

A farmer has access to a pesticide which can be used on corn, soybeans, and wheat fields, and costs \$70/ha-yr to apply. The crop yields the farmer can obtain following crop yields by applying varying rates of pesticides to the field are shown in Table 1.

Application Rate (kg/ha)	Soybean (kg/ha)	Wheat (kg/ha)	Corn (kg/ha)
0	2900	3500	5900
1	3800	4100	6700
2	4400	4200	7900

Table 1: Crop yields from applying varying pesticide rates for Problem

1.

The costs of production, *excluding pesticides*, for each crop, and selling prices, are shown in Table 2.

Crop	Production Cost (\$/ha-yr)	Selling Price (\$/kg)
Soybeans	350	0.36
Wheat	280	0.27
Corn	390	0.22

Table 2: Costs of crop production, excluding pesticides, and selling prices for each crop.

Recently, environmental authorities have declared that farms cannot have an *average* application rate on soybeans, wheat, and corn which exceeds 0.8, 0.7, and 0.6 kg/ha, respectively. The farmer has asked you for advice on how they should plant crops and apply pesticides to maximize profits over 130 total ha while remaining in regulatory compliance if demand for each crop (which is the maximum the market would buy) this year is 250,000 kg?

In this problem:

- Formulate a linear program for this resource allocation problem, including clear definitions of decision variable(s) (including units), objective function(s), and constraint(s) (make sure to

explain functions and constraints with any needed derivations and explanations). **Tip: Make sure that all of your constraints are linear.**

- Implement the program in `JuMP.jl` and find the solution. How many ha should the farmer dedicate to each crop and with what pesticide application rate(s)? How much profit will the farmer expect to make?
- The farmer has an opportunity to buy an extra 10 ha of land. How much extra profit would this land be worth to the farmer? Discuss why this value makes sense and whether you would recommend the farmer should make the purchase.

Solutions

Decision variables: Let $x_{i,j}$ be the amount of land, in ha, dedicated to crop i (soybean, wheat, or corn) using pesticide level j (0, 1, 2)

Parameters

$Y_{i,j}$ = yield in kg/ha, for land used to grow crop i using pesticide level j (given in table)

$ProdC_i$ = production cost for crop i (\$/ha-yr, given in table)

P_i = selling price for crop i (\$/kg, given in table)

L_i = maximum allowed average pesticide application rate for crop i (kg/ha)

maximize total revenue = profit from crops - production costs - pesticide costs

$$\begin{aligned} & \max_{x_{i,j}} \sum_{i \in \mathcal{I}} P_i \sum_{j \in \mathcal{J}} x_{i,j} * Y_{i,j} \\ & - \sum_{i \in \mathcal{I}} ProdC_i \sum_{j \in \mathcal{J}} x_{i,j} - 70 * \sum_{i \in \mathcal{I}} (x_{i,1} + 2 * x_{i,2}) \end{aligned}$$

subject to: $x_{i,j}$

(Pesticides can not exceed limit for each crop)

$\sum_{i \in \mathcal{I}} x_{i,j} \leq L_j$
(Limit on total land)

$\sum_{j \in \mathcal{J}} x_{i,j} \leq D_i$
(Limit on sales based on demand for each crop)

$x_{i,j}$

Creating named matrices with the parameters using `NamedArray`, and solving the model in Julia. The table shows the ha of land for each crop and pesticide level:

Application Rate (kg/ha)	Soybean (ha)	Wheat (ha)	Corn (ha)
0	13.81	6.74	26.92

Application Rate (kg/ha)	Soybean (ha)	Wheat (ha)	Corn (ha)
1	55.25	15.73	0
2	0	0	11.54

From showing the value of the objective function, the farmed can expect \$116,741.17 in profit.

The shadow price for the land constraint is 729.4, so 10 extra ha should increase the profit by about 7,294. Running the model with the land constraint increased to 140 ha, the total profit is now \$124,035.16 confirming that this is 7294 more than the objective function before. Thus, the farmer should purchase the land if it's cost is less than this added profit, and not if it is more than this.

```
In [4]: # @show value.(x);
```

```
In [5]: Y = [2900 3800 4400; 3500 4100 4200; 5900 6700 7900]
PC = [350 280 390]
C = [0.36 0.27 0.22]
```

```
1×3 Matrix{Float64}:
 0.36  0.27  0.22
```

```
In [6]: I = ["S", "W", "C"]
J = ["0", "1", "2"]
```

```
3-element Vector{String}:
 "0"
 "1"
 "2"
```

```
In [7]: Y = NamedArray([2900 3800 4400; 3500 4100 4200; 5900 6700 7900])
```

```
setnames!(Y, I, 1)
setnames!(Y, ["0", "1", "2"], 2)
```

```
(OrderedCollections.OrderedDict{Any, Int64}{"S" => 1, "W" => 2, "C" => 3}, OrderedCollections.OrderedDict{Any, Int64}{"0" => 1, "1" => 2, "2" => 3})
```

```
In [9]: PC = NamedArray([350; 280; 390])
setnames!(PC, I, 1)
```

```
P = NamedArray([0.36; 0.27; 0.22])
setnames!(P, I, 1)
```

```
L = NamedArray([0.8; 0.7; 0.6])
setnames!(L, I, 1)
```

```
(OrderedCollections.OrderedDict{Any, Int64}{"S" => 1, "W" => 2, "C" => 3},)
```

```
In [10]: farm_model = Model(HiGHS.Optimizer) # initialize model object
@variable(farm_model, 0 <= x[i in I, j in J])
@objective(farm_model, Max, sum(P[i]*sum(x[i,j]*Y[i,j] for j = J) for i = I) - sum(PC[i]*
@constraint(farm_model, PestLim[i in I], (x[i,"1"] + 2*x[i,"2"]) <= L[i]*(sum(x[i, j] for j = J) + 1))
@constraint(farm_model, Land, (sum(x[i, j] for i = I for j = J)) == 130)
@constraint(farm_model, Demand[i in I], (sum(x[i, j]*Y[i,j] for j = J) <= 250000))
# @objective(farm_model, Max, sum(P[i]*sum(x[i,j]*Y[i,j] for j = J) for i = I) - sum(PC[i]*sum(x[i,j]*Y[i,j] for j = J) for i = I)
# @constraint(forest_model, Spruce_limit, 0.5*G[1]+G[2]+1.5*G[3] <= 320)
# @constraint(forest_model, Fir_limit, 1.5*G[1]+2*G[2]+2*G[3] <= 720)
# # uncomment the following lines and add the objective and constraints as needed for the forest model
# # @objective(forest_model, )
# # @constraint(forest_model, )
# print(forest_model) # this outputs a nicely formatted summary of the model so you can
```

1-dimensional DenseAxisArray{ConstraintRef{Model, MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64}}, MathOptInterface.LessThan{Float64}}, ScalarShape},1,...} with index sets:

Dimension 1, ["S", "W", "C"]

And data, a 3-element Vector{ConstraintRef{Model, MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64}}, MathOptInterface.LessThan{Float64}}, ScalarShape}}:

Demand[S] : 2900 x[S,0] + 3800 x[S,1] + 4400 x[S,2] <= 250000

Demand[W] : 3500 x[W,0] + 4100 x[W,1] + 4200 x[W,2] <= 250000

Demand[C] : 5900 x[C,0] + 6700 x[C,1] + 7900 x[C,2] <= 250000

```
In [11]: optimize!(farm_model)
```

Running HiGHS 1.7.2 (git hash: 5ce7a2753): Copyright (c) 2024 HiGHS under MIT licence terms

Coefficient ranges:

Matrix [2e-01, 8e+03]

Cost [7e+02, 1e+03]

Bound [0e+00, 0e+00]

RHS [1e+02, 2e+05]

Presolving model

7 rows, 9 cols, 27 nonzeros 0s

6 rows, 8 cols, 28 nonzeros 0s

Presolve : Reductions: rows 6(-1); columns 8(-1); elements 28(+1)

Solving the presolved LP

Using EKK dual simplex solver - serial

Iteration	Objective	Infeasibilities num(sum)
0	-3.7843715799e+01	Ph1: 6(26.7484); Du: 7(37.8437) 0s
5	-1.1674116702e+05	Pr: 0(0) 0s

Solving the original LP from the solution after postsolve

Model status : Optimal

Simplex iterations: 5

Objective value : 1.1674116702e+05

HiGHS run time : 0.02

```
In [12]: @show value.(x);
```

```
value.(x) = 2-dimensional DenseAxisArray{Float64,2,...} with index sets:
  Dimension 1, ["S", "W", "C"]
  Dimension 2, ["0", "1", "2"]
And data, a 3x3 Matrix{Float64}:
13.812154696132593   55.248618784530386   0.0
 6.7433064173395705 15.734381640458993   0.0
26.923076923076923   0.0                               11.538461538461538
```

```
In [13]: @show objective_value(farm_model);
```

```
objective_value(farm_model) = 116741.16702082448
```

```
In [14]: @show shadow_price(Land);
```

```
shadow_price(Land) = 729.4
```

10 more ha

```
In [15]: farm_model = Model(HiGHS.Optimizer) # initialize model object
@variable(farm_model, 0 <= x[i in I, j in J])
@objective(farm_model, Max, sum(P[i]*sum(x[i,j]*Y[i,j] for j = J) for i = I) - sum(PC[i]*
@constraint(farm_model, PestLim[i in I], (x[i,"1"] + 2*x[i,"2"]) <= L[i]*(sum(x[i, j] for j = J)))
@constraint(farm_model, Land, (sum(x[i, j] for i = I for j = J)) == 140)
@constraint(farm_model, Demand[i in I], (sum(x[i, j]*Y[i,j] for j = J) <= 250000))
# @objective(farm_model, Max, sum(P[i]*sum(x[i,j]*Y[i,j] for j = J) for i = I) - sum(PC[i]*
# @constraint(forest_model, Spruce_limit, 0.5*G[1]+G[2]+1.5*G[3] <= 320)
# @constraint(forest_model, Fir_limit, 1.5*G[1]+2*G[2]+2*G[3] <= 720)
# # uncomment the following lines and add the objective and constraints as needed for the forest model
# # @objective(forest_model, )
# # @constraint(forest_model, )
# print(forest_model) # this outputs a nicely formatted summary of the model so you can
```

```
1-dimensional DenseAxisArray{ConstraintRef{Model, MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64}, MathOptInterface.LessThan{Float64}}, ScalarShape},1,...} with index sets:
  Dimension 1, ["S", "W", "C"]
And data, a 3-element Vector{ConstraintRef{Model, MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64}, MathOptInterface.LessThan{Float64}}, ScalarShape}}:
Demand[S] : 2900 x[S,0] + 3800 x[S,1] + 4400 x[S,2] <= 250000
Demand[W] : 3500 x[W,0] + 4100 x[W,1] + 4200 x[W,2] <= 250000
Demand[C] : 5900 x[C,0] + 6700 x[C,1] + 7900 x[C,2] <= 250000
```

```
In [16]: optimize!(farm_model)
```

Running HiGHS 1.7.2 (git hash: 5ce7a2753): Copyright (c) 2024 HiGHS under MIT licence terms

Coefficient ranges:

```
Matrix [2e-01, 8e+03]
Cost   [7e+02, 1e+03]
Bound  [0e+00, 0e+00]
RHS    [1e+02, 2e+05]
```

Presolving model

7 rows, 9 cols, 27 nonzeros 0s

6 rows, 8 cols, 28 nonzeros 0s

Presolve : Reductions: rows 6(-1); columns 8(-1); elements 28(+1)

Solving the presolved LP

Using EKK dual simplex solver - serial

```
Iteration      Objective      Infeasibilities num(sum)
      0      -3.7843715799e+01 Ph1: 6(26.7484); Du: 7(37.8437) 0s
      5      -1.2403516702e+05 Pr: 0(0) 0s
```

Solving the original LP from the solution after postsolve

Model status : Optimal

Simplex iterations: 5

Objective value : 1.2403516702e+05

HiGHS run time : 0.00

In [17]: @show value.(x);

value.(x) = 2-dimensional DenseAxisArray{Float64,2,...} with index sets:

Dimension 1, ["S", "W", "C"]

Dimension 2, ["0", "1", "2"]

And data, a 3×3 Matrix{Float64}:

```
13.812154696132593  55.248618784530386  0.0
 9.743306417339566  22.73438164045898  0.0
26.923076923076923  0.0                11.538461538461538
```

In [18]: @show objective_value(farm_model);

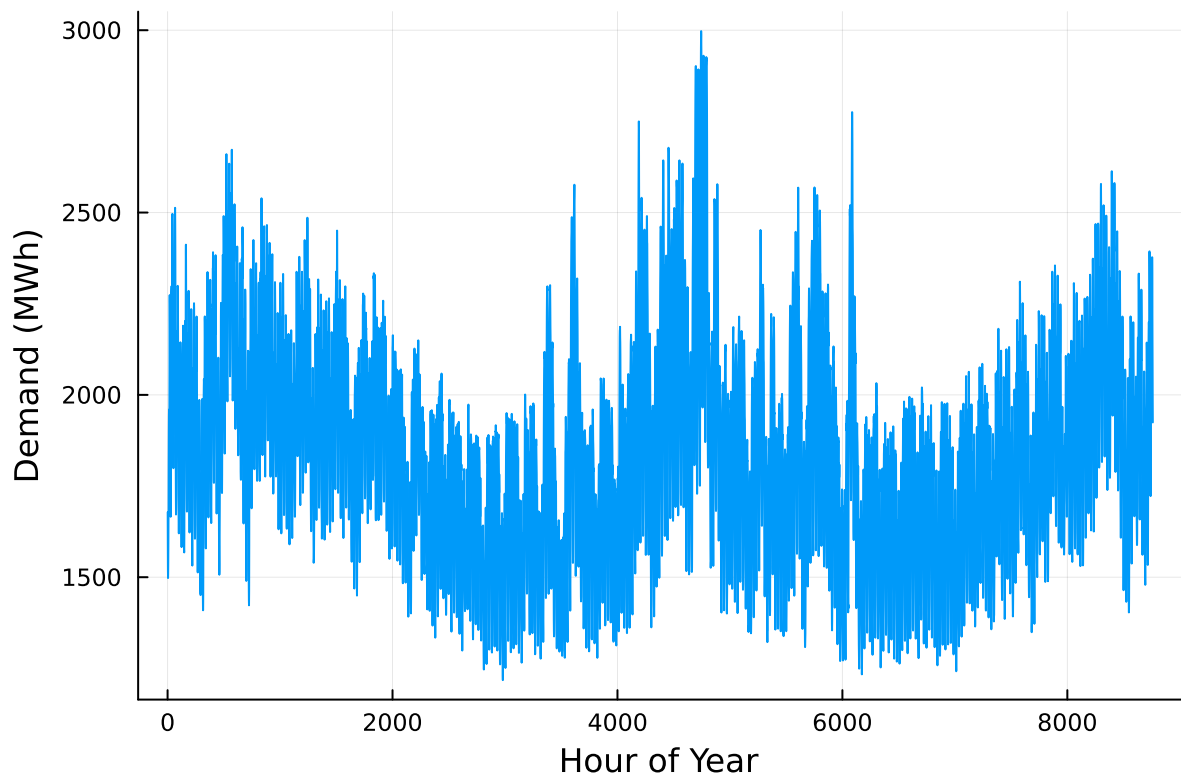
objective_value(farm_model) = 124035.16702082448

Problem 2 (30 points)

For this problem, we will use hourly load (demand) data from 2013 in New York's Zone C (which includes Ithaca). The load data is loaded and plotted below in Figure 1.

```
In [19]: # Load the data, pull Zone C, and reformat the DataFrame
NY_demand = DataFrame(CSV.File("data/2013_hourly_load_NY.csv"))
rename!(NY_demand, :Time Stamp => :Date)
demand = NY_demand[:, [:Date, :C]]
rename!(demand, :C => :Demand)
demand[:, :Hour] = 1:nrow(demand)

# plot demand
plot(demand.Hour, demand.Demand, xlabel="Hour of Year", ylabel="Demand (MWh)", label=:f
```



Next, we load the generator data, shown in Table 3. This data includes fixed costs ($/MWh_{installed}$), $variablecosts$ ($/MWh$ generated), and CO_2 emissions intensity (tCO_2/MWh generated).

```
In [20]: gens = DataFrame(CSV.File("data/generators.csv"))
```

6×4 DataFrame

Row	Plant	FixedCost	VarCost	Emissions
	String15	Int64	Int64	Float64
1	Geothermal	450000	0	0.0
2	Coal	220000	24	1.0
3	NG CCGT	82000	30	0.43
4	NG CT	65000	40	0.55
5	Wind	91000	0	0.0
6	Solar	70000	0	0.0

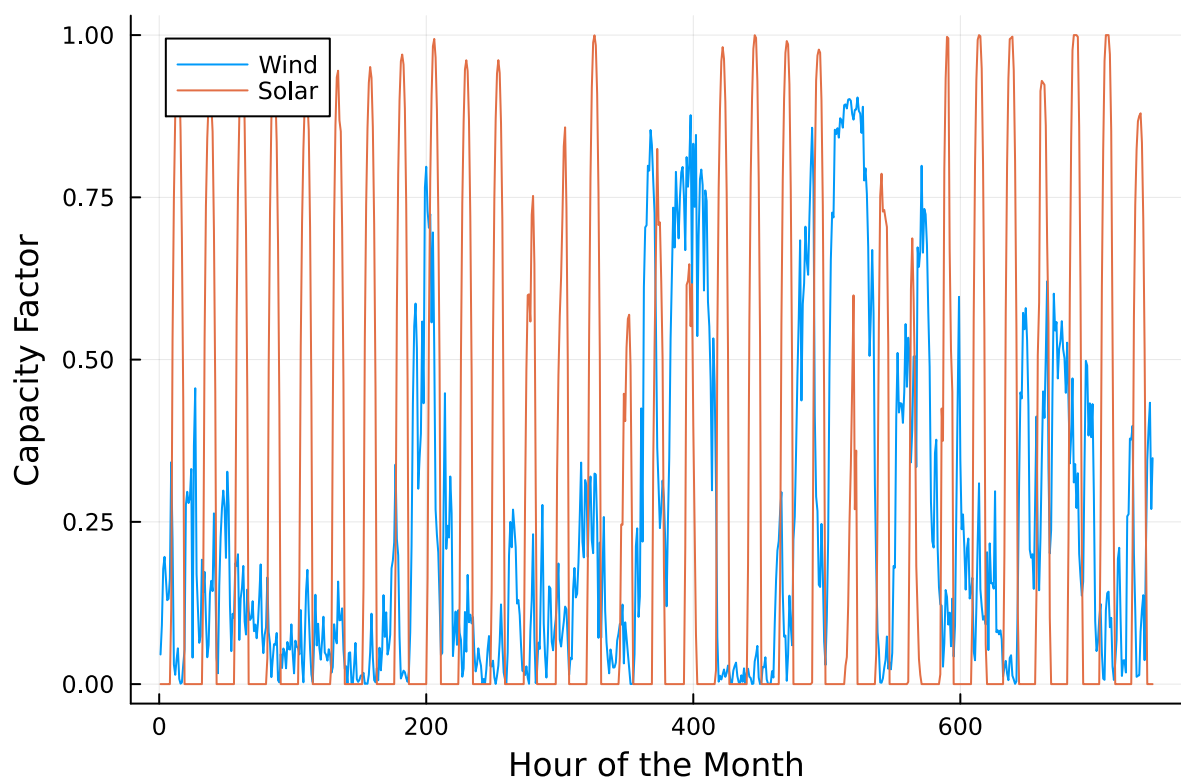
Finally, we load the hourly solar and wind capacity factors, which are plotted in Figure 2. These tell us the fraction of installed capacity which is expected to be available in a given hour for generation (typically based on the average meteorology).

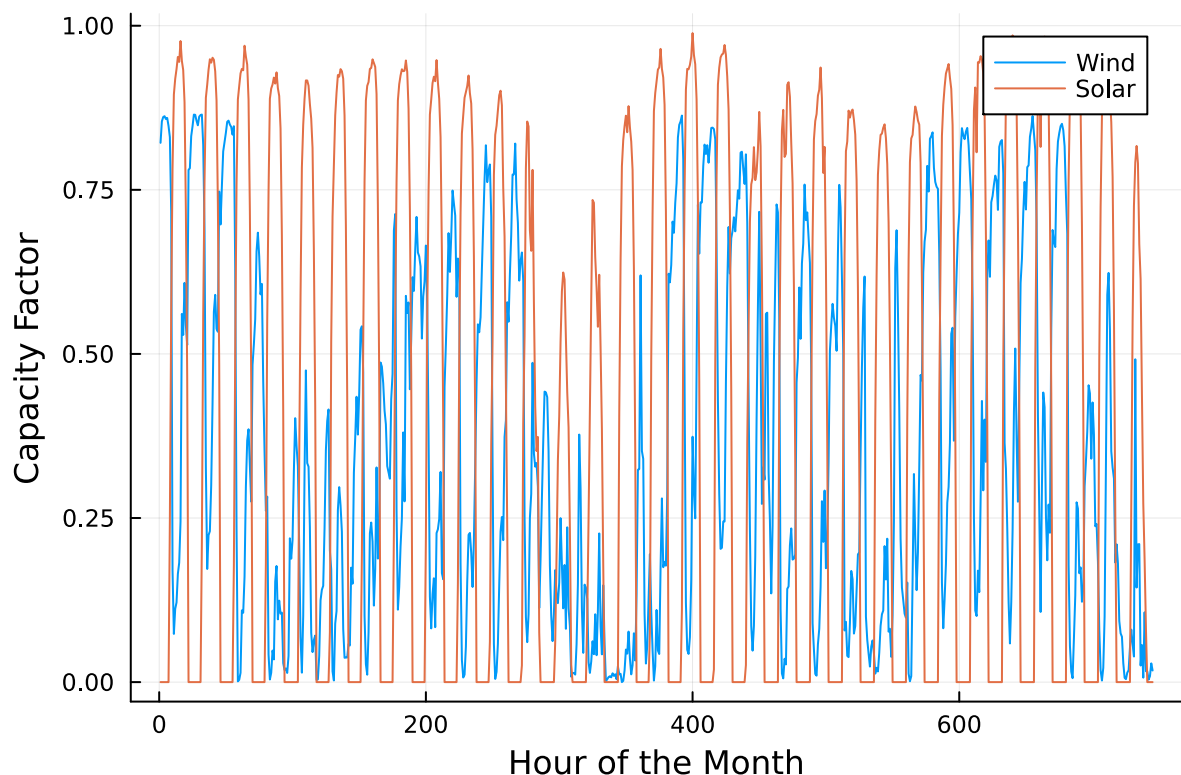

```
In [21]: # Load capacity factors into a DataFrame
cap_factor = DataFrame(CSV.File("data/wind_solar_capacity_factors.csv"))

# plot January capacity factors
p1 = plot(cap_factor.Wind[1:(24*31)], label="Wind")
plot!(cap_factor.Solar[1:(24*31)], label="Solar")
xaxis!("Hour of the Month")
yaxis!("Capacity Factor")

p2 = plot(cap_factor.Wind[4344:4344+(24*31)], label="Wind")
plot!(cap_factor.Solar[4344:4344+(24*31)], label="Solar")
xaxis!("Hour of the Month")
yaxis!("Capacity Factor")

display(p1)
display(p2)
```





You have been asked to develop a generating capacity expansion plan for the utility in Riley County, NY, which currently has no existing electrical generation infrastructure. The utility can build any of the following plant types: geothermal, coal, natural gas combined cycle gas turbine (CCGT), natural gas combustion turbine (CT), solar, and wind.

While coal, CCGT, and CT plants can generate at their full installed capacity, geothermal plants operate at maximum 85% capacity, and solar and wind available capacities vary by the hour depend on the expected meteorology. The utility will also penalize any non-served demand at a rate of \$10,000/MWh.

In this problem:

- Formulate a linear program for this capacity expansion problem, including clear definitions of decision variable(s) (including units), objective function(s), and constraint(s) (make sure to explain functions and constraints with any needed derivations and explanations).
- Implement your linear program in `JuMP.jl`. Find the optimal solution. How much should the utility build of each type of generating plant? What will the total cost be? How much energy will be non-served?
- What fraction of annual generation does each plant type produce? How does this compare to the breakdown of built capacity that you found in Problem 1.5? Do these results make sense given the generator data?
- Make a plot of the electricity price in each hour. Discuss any trends that you see.

Significant Digits

Use `round(x; digits=n)` to report values to the appropriate precision! If your number is on a different order of magnitude and you want to round to a certain number of significant digits, you can use `round(x; sigdigits=n)`.

Getting Variable Output Values

`value.(x)` will report the values of a JuMP variable `x`, but it will return a special container which holds other information about `x` that is useful for JuMP. This means that you can't use this output directly for further calculations. To just extract the values, use `value.(x).data`.

Suppressing Model Command Output

The output of specifying model components (variable or constraints) can be quite large for this problem because of the number of time periods. If you end a cell with an `@variable` or `@constraint` command, I *highly* recommend suppressing output by adding a semi-colon after the last command, or you might find that your notebook crashes.

Solutions

Decision variables: x_g = Installed capacity for generator type g (MW)

$y_{g,t}$ = Production from generator g in time t (MWh)

NSE_t = non-served energy in time t (MWh)

Minimize total cost = fixed cost + variable cost + unserved energy cost

$$\min_{x,y,NSE} \sum_{g \in \mathcal{G}} \text{Fixed} + \sum_{t \in \mathcal{T}}$$

$$\text{subject to: } \sum_{g \in \mathcal{G}} y_{g,t} +$$

(Meeting demand in each hour)

$$y_{g,t} \leq x_g$$

(Generator capacity limits)

$$x_g, y_{g,t}, NSE_t \geq 0$$

- Implement your linear program in `JuMP.jl`. Find the optimal solution. How much should the utility build of each type of generating plant? What will the total cost be? How much energy will be non-served?

Type	Capacity (MW)
Geothermal	0
Coal	0
NG CCGT	1658.45
NG CT	879.75
Wind	484.95
Solar	1957.64

Total cost = \$654.58 M

Nonserved load = 256.83 MWh

- What fraction of annual generation does each plant type produce? How does this compare to the breakdown of built capacity that you found in Problem 1.5? Do these results make sense given the generator data?

Type	Percent Capacity	Percent Annual Generation
Geothermal	0%	0%
Coal	0%	0%
NG CCGT	33.30%	52.89%
NG CT	17.66%	2.74%
Wind	9.74%	8.72%
Solar	39.30%	35.65%

These results do make sense. NG CCGTs, with their relatively high capacity factor, produce over 50% of total power, despite being only 33.3% of installed capacity. While NG CTs can function at high capacity factors, due to their high variable cost they tend to act as peaker plants, used to meet demand during times when it is at its maximum (and when wind and solar generation are low). Thus, it makes sense that such plants make up only a small percent of total generation. Because wind and solar have lower capacity factors compared to gas due to their weather dependence, they make up a slightly smaller portion of generation than they do of capacity.

- Make a plot of the electricity price in each hour. Discuss any trends that you see.

The electricity price is highly peaked. A few times throughout the year, the price spikes to \$1000/MWh, during periods with unserved load. Higher prices occur during the beginning and end of the year, corresponding to winter hours when solar resources are poor, and electricity demand is relatively high due to increased lighting and heating requirements (in the case of heat pumps or resistive heating). The prices are also higher during the summer time (around hours 4000-6000), corresponding to peak electricity demand during this season. During the spring and fall, when temperatures are less extreme, the prices are lower due to lower electricity demand for climate control.

```
In [22]: cap_factor[:, :Coal] .=1
cap_factor[:, :Geothermal] .=0.8
cap_factor[:, :NG_CCGT] .=1
cap_factor[:, :NG_CT] .=1
select!(cap_factor, :Geothermal, :Coal, :NG_CCGT, :NG_CT, :Wind, :Solar)
```

8760×6 DataFrame

8735 rows omitted

Row	Geothermal	Coal	NG_CCGT	NG_CT	Wind	Solar
	Float64	Int64	Int64	Int64	Float64	Float64
1	0.8	1	1	1	0.0456225	0.0
2	0.8	1	1	1	0.0908019	0.0
3	0.8	1	1	1	0.177524	0.0
4	0.8	1	1	1	0.196106	0.0
5	0.8	1	1	1	0.163481	0.0
6	0.8	1	1	1	0.129312	0.0
7	0.8	1	1	1	0.131448	0.0
8	0.8	1	1	1	0.162792	0.0
9	0.8	1	1	1	0.341911	0.115556
10	0.8	1	1	1	0.222848	0.530593
11	0.8	1	1	1	0.030222	0.747114
12	0.8	1	1	1	0.0143295	0.874793
13	0.8	1	1	1	0.0377381	0.94319
⋮	⋮	⋮	⋮	⋮	⋮	⋮
8749	0.8	1	1	1	0.284942	0.734762
8750	0.8	1	1	1	0.462101	0.643106
8751	0.8	1	1	1	0.327947	0.632685
8752	0.8	1	1	1	0.249119	0.560314
8753	0.8	1	1	1	0.171991	0.445911
8754	0.8	1	1	1	0.234215	0.232936
8755	0.8	1	1	1	0.32073	0.0
8756	0.8	1	1	1	0.166379	0.0
8757	0.8	1	1	1	0.252252	0.0
8758	0.8	1	1	1	0.276054	0.0
8759	0.8	1	1	1	0.111131	0.0
8760	0.8	1	1	1	0.208158	0.0

```

In [23]: # define sets
G = 1:nrow(gens)
T = 1:nrow(demand)
NSECost = 10000

gencap = Model(HiGHS.Optimizer)
# define variables
@variables(gencap, begin
    x[g in G] >= 0
    y[g in G, t in T] >= 0
    NSE[t in T] >= 0
end)

gens
gens[1, :FixedCost]
@objective(gencap, Min,
    sum(gens[G, :FixedCost] .* x) + sum(gens[G, :VarCost] .* sum(y[:, t] for t in T)) +
)

@constraint(gencap, load[t in T], sum(y[:, t]) + NSE[t] >= demand.Demand[t])

@constraint(gencap, availability[g in G, t in T], y[g, t] <= x[g]*cap_factor[t,g])
optimize!(gencap)

```

Running HiGHS 1.7.2 (git hash: 5ce7a2753): Copyright (c) 2024 HiGHS under MIT licence terms

Coefficient ranges:

```

Matrix [5e-05, 1e+00]
Cost   [2e+01, 4e+05]
Bound  [0e+00, 0e+00]
RHS    [1e+03, 3e+03]

```

Presolving model

56856 rows, 56862 cols, 153048 nonzeros 1s

56853 rows, 56859 cols, 153042 nonzeros 2s

Presolve : Reductions: rows 56853(-4467); columns 56859(-4467); elements 153042(-8934)

Solving the presolved LP

Using EKK dual simplex solver - serial

Iteration	Objective	Infeasibilities	num(sum)
0	0.0000000000e+00	Pr: 8760(4.1747e+06)	4s
26884	4.5608792068e+08	Pr: 6262(592480); Du: 0(5.00188e-06)	9s
37348	6.2141093315e+08	Pr: 4671(617695); Du: 0(6.08798e-07)	14s
39418	6.4462975333e+08	Pr: 3493(793282); Du: 0(5.67463e-07)	20s
40997	6.5393055344e+08	Pr: 1328(51092.7); Du: 0(2.14312e-06)	25s
42008	6.5458487038e+08	Pr: 0(0); Du: 0(2.07029e-10)	29s

Solving the original LP from the solution after postsolve

```

Model status      : Optimal
Simplex iterations: 42008
Objective value    : 6.5458487038e+08
HiGHS run time     : 28.98

```

```

In [24]: @show objective_value(gencap);

objective_value(gencap) = 6.54584870381541e8

```

```

In [25]: @show value.(x);

```

```

value.(x) = 1-dimensional DenseAxisArray{Float64,1,...} with index sets:
  Dimension 1, 1:6
And data, a 6-element Vector{Float64}:
 -0.0
  0.0
1658.4514133074133
 879.7538120149645
484.946167110047
1957.6424239086684

```

```
In [26]: [@show value(sum(x[i])/sum(x)) for i in 1:6]
```

```

value(sum(x[i]) / sum(x)) = -0.0
value(sum(x[i]) / sum(x)) = 0.0
value(sum(x[i]) / sum(x)) = 0.3329692965539611
value(sum(x[i]) / sum(x)) = 0.17662923711651135
value(sum(x[i]) / sum(x)) = 0.09736322863215606
value(sum(x[i]) / sum(x)) = 0.3930382376973714
6-element Vector{Float64}:
 -0.0
  0.0
 0.3329692965539611
 0.17662923711651135
 0.09736322863215606
 0.3930382376973714

```

```
In [27]: [@show value(sum(y[i, t] for t = T)/sum(y)) for i in 1:6]
```

```

value(sum((y[i, t] for t = T)) / sum(y)) = 0.0
value(sum((y[i, t] for t = T)) / sum(y)) = 0.0
value(sum((y[i, t] for t = T)) / sum(y)) = 0.5288876275987046
value(sum((y[i, t] for t = T)) / sum(y)) = 0.027455127324108597
value(sum((y[i, t] for t = T)) / sum(y)) = 0.08719666131388551
value(sum((y[i, t] for t = T)) / sum(y)) = 0.3564605837632996
6-element Vector{Float64}:
 0.0
 0.0
 0.5288876275987046
 0.027455127324108597
 0.08719666131388551
 0.3564605837632996

```

```
In [28]: @show value(sum(NSE));
```

```
value(sum(NSE)) = 256.83377442738583
```

```
In [ ]: elec_price = @show shadow_price.(load)
```

```
shadow_price.(load) = 1-dimensional DenseAxisArray{Float64,1,...} with index sets:
```

```
Dimension 1, 1:8760
```

```
And data, a 8760-element Vector{Float64}:
```

```
-30.0  
-30.0  
-30.0  
-30.0  
-30.0  
-30.0  
-30.0  
-30.0  
-30.0  
-30.0  
-30.0  
-30.0  
-30.0  
-30.0  
-30.0  
-30.0  
-30.0  
-30.0  
-40.0  
-40.0  
-40.0  
-40.0  
-40.0  
-30.0  
-40.0  
-30.0  
-30.0  
-30.0  
-40.0  
-40.0  
-40.0  
-40.0  
-40.0  
-40.0  
-30.0  
-30.0  
-30.0  
-30.0  
-30.0  
-30.0  
-30.0  
-40.0  
-40.0  
-40.0  
-40.0  
-40.0  
-40.0  
-40.0  
-40.0  
-40.0  
-30.0  
-40.0  
-40.0  
-40.0
```


-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0

-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
0.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0

-40.0
-30.0
-40.0
-30.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0

-30.0

-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0

-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0

-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0

-30.0

-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-37.99030581357043
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-10000.0
-10000.0
-10000.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-40.0

-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0

-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0

[illegible]

-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0

[illegible]

-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0

-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0

-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0

-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0

-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-30.0

[illegible]

-30.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0

-30.0

-40.0

-30.0

0.0

0.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0

-40.0
-32.00969418642957
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-30.0
-30.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0

-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
-30.0

-30.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0

-30.0
-30.0
-30.0
-30.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0

-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
-30.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-40.0
-40.0

-30.0

-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0

-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
-30.0
-30.0
-30.0
0.0
0.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-30.0
-30.0

-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0

-30.0
-30.0
-30.0
0.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
-30.0
-30.0

-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0

[illegible]

-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
-30.0
-30.0
-30.0
-30.0

-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0

-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0

[illegible]

-30.0

-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
-30.0

-30.0

-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0

0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-30.0

[illegible]

-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
-30.0
0.0
0.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0

0.0
0.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-30.0
-40.0
-30.0
-30.0
-30.0

-30.0

0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
-30.0
0.0
-30.0
0.0
-30.0
0.0
-30.0

-30.0
-30.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0

-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0

0.0
0.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
-30.0
0.0
-30.0
-30.0
-30.0
-40.0

[illegible]

-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
0.0

-30.0

[illegible]

-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0

-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
-30.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0

-30.0

[illegible]

-30.0

-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-19.842896938122443
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
0.0

0.0
0.0
0.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0

[illegible]

[illegible]

-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0

[illegible]

-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
-30.0
0.0
-30.0
-30.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0

[illegible]

[illegible]

[illegible]

[illegible]

-30.0

-30.0

-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0

-40.0

-30.0

-30.0

-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-10000.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0

-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0

[illegible]

-30.0

[illegible]

-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
-30.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-30.0
-30.0

-30.0

-30.0

[illegible]

-30.0

-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0

[illegible]

[illegible]

[illegible]

-30.0

-30.0

[illegible]

-30.0

[illegible]

-30.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0

-30.0

-40.0

[illegible]

[illegible]

-40.0

[illegible]

0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
-30.0

-30.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
-30.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0

0.0
0.0

-40.0

-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
0.0
-30.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0

-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
0.0

-30.0

[illegible]

-30.0
-30.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-30.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
-30.0

[illegible]

-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
-30.0
0.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0

135/171

-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
-30.0
0.0
0.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
-30.0
0.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0

-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
-30.0
0.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
-30.0
0.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
-30.0

0.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
-30.0
0.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0

[illegible]

[illegible]

-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0

-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0

0.0
0.0
0.0
0.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0

-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0

-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
0.0

0.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0

-30.0

-30.0
0.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0

-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0

-30.0

-40.0

-30.0

-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0

-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0

-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0

-40.0

-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0

-30.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0

-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0

-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-40.0
-40.0

-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-10000.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0

-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-40.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0

-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-10000.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0

-30.0

-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0

-30.0

-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0

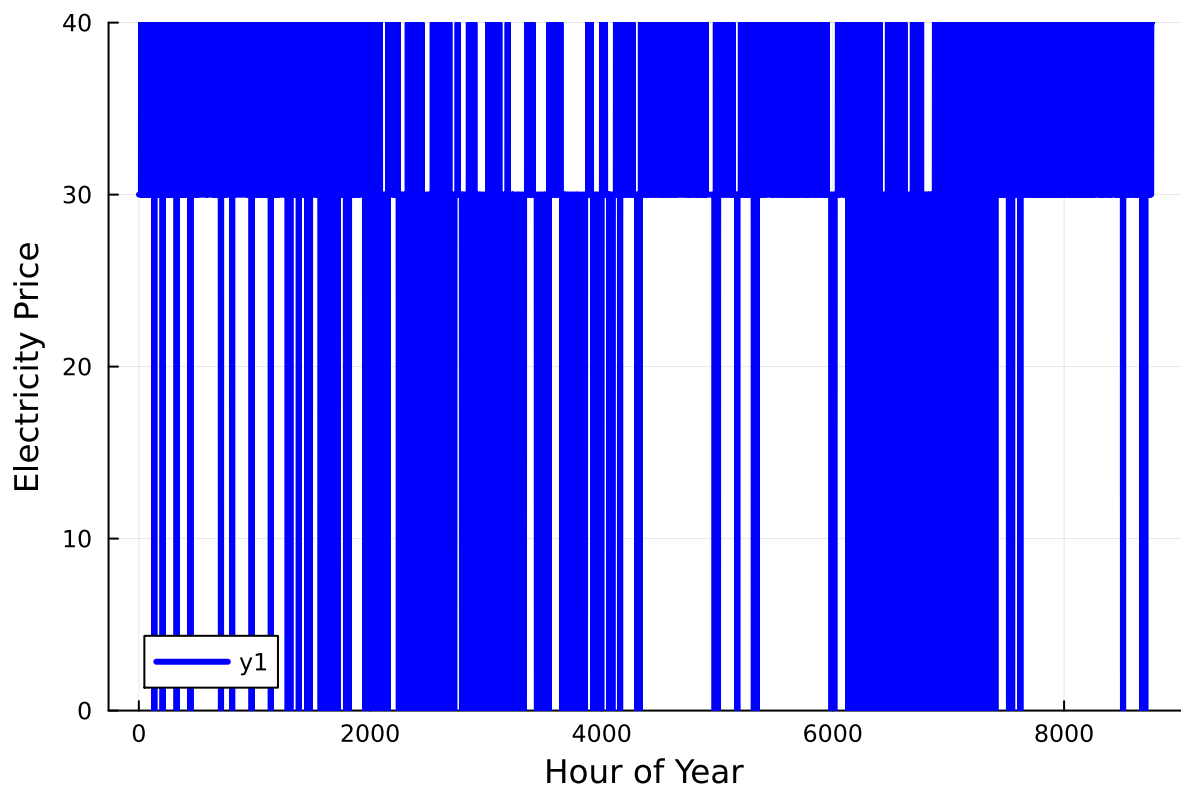
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
0.0
0.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
-30.0
0.0
-30.0
-30.0
-30.0
-30.0
-40.0
-40.0
-40.0
-40.0
-40.0
-30.0
-30.0
-30.0
-30.0
-40.0

-40.0
 -40.0
 -40.0
 -30.0
 -30.0
 -30.0
 -30.0
 -30.0
 -30.0
 -30.0
 -30.0
 -40.0
 -40.0
 -40.0
 -40.0
 -40.0
 -40.0
 -40.0
 -40.0
 -30.0
 -30.0
 -30.0
 -40.0
 -30.0
 -40.0
 -40.0
 -30.0
 -30.0
 -30.0
 -30.0
 -30.0
 -30.0
 -30.0
 -40.0
 -40.0
 -40.0
 -40.0
 -40.0
 -40.0
 -40.0

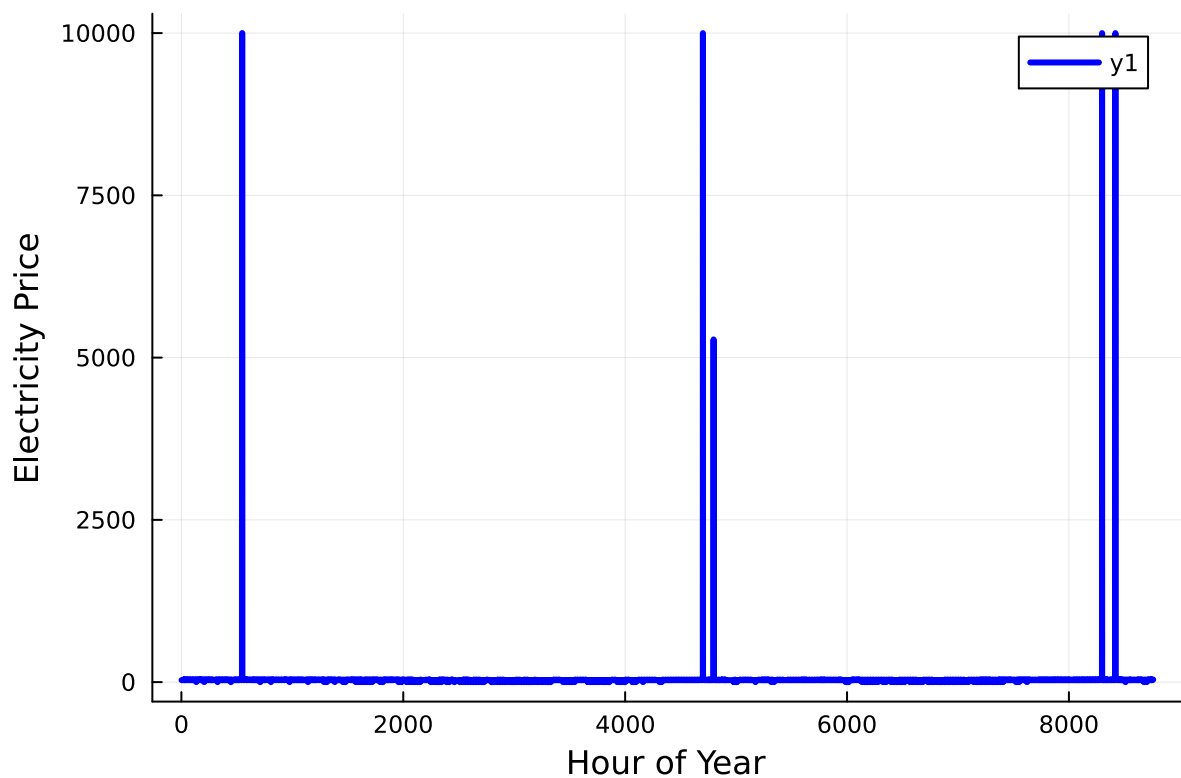
```

In [32]: p = plot(; ylim = (0, 40), xlabel="Hour of Year", ylabel="Electricity Price")

# p = plot(; xlabel="Time", ylabel=["The function ", L"y = 2x + 1", " is\\affine and no
# add simulated solution
plot!(p, T, -elec_price.data, linewidth=3, color=:blue)
  
```



```
In [33]: p = plot(; xlabel="Hour of Year", ylabel="Electricity Price")
# p = plot(; xlabel="Time", ylabel=["The function ", L" $y = 2x + 1$ ", " is\\affine and no
# add simulated solution
plot!(p, T, -elec_price.data, linewidth=3, color=:blue)
```



Problem 3 (10 points)

This problem is only required for students in BEE 5750.

The NY state legislature is considering enacting an annual CO₂ limit, which for the utility would limit the emissions in its footprint to 1.5 MtCO₂/yr.

In this problem:

- Reformulate your linear program from Problem 2 with any necessary changes to capture the CO₂ limit.
- Implement the new optimization problem and find the optimal solution. How much should the utility build of each type of generating plant? What is different from your plan from Problem 1? Do these changes make sense?
- What would the value to the utility be of allowing it to emit an additional 1000 tCO₂/yr? An additional 5000?

References

List any external references consulted, including classmates.

https://penzai.readthedocs.io/en/v0.1.3/_autosummary/leaf/penzai.core.named_axes.NamedArray.html
(https://penzai.readthedocs.io/en/v0.1.3/_autosummary/leaf/penzai.core.named_axes.NamedArray.html)

https://dataframes.juliadata.org/stable/man/getting_started/
(https://dataframes.juliadata.org/stable/man/getting_started/)

<https://jump.dev/JuMP.jl/stable/manual/containers/#Access-internal-data>
(<https://jump.dev/JuMP.jl/stable/manual/containers/#Access-internal-data>)

